

Phase 9 Structured Note: Quadratic Dividing Work and Division Master Theorem

1. Current Progress Analysis

You are now in **Phase 9**.

Before Phase 9, you learned how to write and solve many recurrence relations.

Phase	Main idea	Example recurrence	Final result
Phase 1	Turn recursive code into a recurrence relation	$T(n) = T(n - 1) + 1$	$O(n)$
Phase 2	Solve one-call decreasing recurrences	$T(n) = T(n - 1) + n$	$O(n^2)$
Phase 3	Handle two decreasing recursive calls	$T(n) = 2T(n - 1) + 1$	$O(2^n)$
Phase 4	Use the decreasing master theorem	$T(n) = aT(n - b) + f(n)$	shortcut
Phase 5	One half-size call with constant work	$T(n) = T(n / 2) + 1$	$O(\log n)$
Phase 6	One half-size call with linear work	$T(n) = T(n / 2) + n$	$O(n)$
Phase 7	Two half-size calls with constant work	$T(n) = 2T(n / 2) + 1$	$O(n)$
Phase 8	Two half-size calls with linear work	$T(n) = 2T(n / 2) + n$	$O(n \log n)$

Big idea from Phase 8

In Phase 8, the recurrence was:

$$T(n) = 2T(n / 2) + n$$

This means:

two recursive calls on half the input
+ linear work at each call

The answer was:

$O(n \log n)$

Why?

Because each level costs n , and there are $\log n$ levels.

So:

$n \times \log n = n \log n$

What changes in Phase 9?

In Phase 9, the recursive calls stay the same:

$2T(n / 2)$

But the extra work changes from linear to quadratic.

Phase 8:

$T(n) = 2T(n / 2) + n$

Phase 9:

$T(n) = 2T(n / 2) + n^2$

The only change is:

$+n$ changes to $+n^2$

That change is very important.

It changes the final answer from:

$O(n \log n)$

to:

$O(n^2)$

2. Main Goal of Phase 9

The goal of Phase 9 is to understand recursive functions that:

1. Do **quadratic work** in the current call.
2. Make **two recursive calls**.
3. Give each recursive call **half of the input**.

The main recurrence is:

$$T(n) = 2T(n / 2) + n^2$$

The final time complexity is:

$O(n^2)$

The stack space is:

$O(\log n)$

Main lesson

Two half-size recursive calls plus quadratic work gives:

$O(n^2)$

This happens because the quadratic work is so large that it dominates the total cost.

3. What Is Quadratic Dividing Work?

A **dividing function** is a recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

This means the next recursive call receives half of the current input.

In Phase 9, the function makes two recursive calls:

```
algorithm(n / 2);  
algorithm(n / 2);
```

So the recursive part is:

```
2T(n / 2)
```

The function also does quadratic work using nested loops:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << i << " ";  
    }  
}
```

The outer loop runs n times.

The inner loop also runs n times for each outer-loop step.

So the total loop work is:

```
n × n = n2
```

Therefore, the full recurrence is:

```
T(n) = 2T(n / 2) + n2
```

4. Main Code Example

```
#include <iostream>
using namespace std;

void algorithm(int n) {
    if (n > 1) {
        // Quadratic work:  $O(n^2)$ 
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                cout << i << " ";
            }
        }

        cout << "| ";

        // Two recursive calls, each on half of n
        algorithm(n / 2);
        algorithm(n / 2);
    }
}

int main() {
    int n = 4;
    cout << "Output: ";
    algorithm(n);
    cout << endl;
    return 0;
}
```

5. What the Code Does

The function is:

```
void algorithm(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                cout << i << " ";
            }
        }

        algorithm(n / 2);
    }
}
```

```
        algorithm(n / 2);
    }
}
```

It does four main things:

1. It checks whether `n > 1`.
2. If true, it runs nested loops.
3. It calls itself once with `n / 2`.
4. It calls itself again with `n / 2`.

The function stops when `n` becomes `1` or smaller.

6. Breaking the Code Into Parts

Look at the important part:

```
if (n > 1) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << i << " ";
        }
    }

    algorithm(n / 2);
    algorithm(n / 2);
}
```

Inside each working call, there are four important parts.

Part 1: Base Condition

```
if (n > 1)
```

This means the function continues only while `n` is greater than `1`.

When `n = 1`, the condition becomes false:

```
1 > 1 is false
```

So the function stops.

The base case is:

$$T(1) = 1$$

This means the stopping case takes constant time.

Part 2: Current Work

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << i << " ";  
    }  
}
```

This is a nested loop.

The outer loop runs times.

The inner loop runs times for each outer-loop run.

So the total work is:

$$n \times n = n^2$$

This is called **quadratic work**.

So:

$$f(n) = n^2$$

Part 3: First Recursive Call

```
algorithm(n / 2);
```

This call gives half of the input to the next recursive call.

Its time is:

$T(n / 2)$

Part 4: Second Recursive Call

`algorithm(n / 2);`

This is another recursive call with the same input size.

Its time is also:

$T(n / 2)$

So the two recursive calls together give:

$T(n / 2) + T(n / 2)$

That simplifies to:

$2T(n / 2)$

7. Writing the Recurrence Relation

The total time is:

first recursive call time
+ second recursive call time
+ current work

The first recursive call is:

$T(n / 2)$

The second recursive call is:

$$T(n / 2)$$

The current work is:

$$n^2$$

So:

$$T(n) = T(n / 2) + T(n / 2) + n^2$$

Since the two recursive calls are the same size:

$$T(n) = 2T(n / 2) + n^2$$

Base case:

$$T(1) = 1$$

Full recurrence:

$$T(n) = 2T(n / 2) + n^2, \text{ for } n > 1$$
$$T(1) = 1$$

8. Meaning of Each Part of the Recurrence

The recurrence is:

$$T(n) = 2T(n / 2) + n^2$$

Part	Meaning
$T(n)$	Time taken for input size n
2	There are two recursive calls
$T(n / 2)$	Each recursive call receives half the input
$+ n^2$	Quadratic work done in the current call

Part	Meaning
$T(1) = 1$	Base case takes constant time

In simple words:

The function does n^2 work now,
then creates two smaller problems,
each of size $n / 2$.

9. General Division Recurrence Form

Dividing recurrences often follow this form:

$$T(n) = aT(n / b) + f(n)$$

Where:

Symbol	Meaning
a	number of recursive calls
b	division factor
$f(n)$	extra work outside recursion

For Phase 9:

$$T(n) = 2T(n / 2) + n^2$$

So:

Symbol	Value	Reason
a	2	There are two recursive calls
b	2	The input is divided by 2
$f(n)$	n^2	The nested loops run $n \times n$ times

10. Dry Run for `algorithm(4)`

Call:

```
algorithm(4)
```

Step 1: First call, `algorithm(4)`

Since `4 > 1` is true, the nested loops run.

The nested loops do:

```
4 × 4 = 16
```

So the first call does `16` units of work.

Then it calls:

```
algorithm(2)  
algorithm(2)
```

Step 2: First `algorithm(2)`

Since `2 > 1` is true, the nested loops run.

The nested loops do:

```
2 × 2 = 4
```

So the first `algorithm(2)` does `4` units of work.

Then it calls:

```
algorithm(1)  
algorithm(1)
```

Both `algorithm(1)` calls stop because:

$1 > 1$ is false

Step 3: Second `algorithm(2)`

The second `algorithm(2)` also does:

$2 \times 2 = 4$

Then it calls:

```
algorithm(1)
algorithm(1)
```

Both stop.

Work table for `n = 4`

Level	Number of calls	Size of each call	Work per call	Total work at level
Level 0	1	4	$4^2 = 16$	16
Level 1	2	2	$2^2 = 4$	$2 \times 4 = 8$
Stop level	4	1	stop	ignored in Big O

Total work:

$16 + 8 = 24$

11. Recursion Tree Intuition

For `n = 4`, the recursion tree looks like this:

```
      4
     /
    2  2
```



The 1 calls stop because they reach the base case.

Each working node does work equal to the square of its input size.

So:

Level 0: one call of size 4
 Work = $4^2 = 16$

Level 1: two calls of size 2
 Work = $2^2 + 2^2 = 4 + 4 = 8$

The total is:

$$16 + 8 = 24$$

12. Level-by-Level Intuition for General n

For general input n , the recursion tree works like this:

Level	Number of calls	Size of each call	Work per call	Total work at level
Level 0	1	n	n^2	n^2
Level 1	2	$n / 2$	$(n / 2)^2 = n^2 / 4$	$2 \times n^2 / 4 = n^2 / 2$
Level 2	4	$n / 4$	$(n / 4)^2 = n^2 / 16$	$4 \times n^2 / 16 = n^2 / 4$
Level 3	8	$n / 8$	$(n / 8)^2 = n^2 / 64$	$8 \times n^2 / 64 = n^2 / 8$
Level i	2^i	$n / 2^i$	$(n / 2^i)^2$	$n^2 / 2^i$

Key idea

The total work per level keeps getting smaller:

$n^2, n^2/2, n^2/4, n^2/8, \dots$

This is very different from Phase 8.

In Phase 8, every level cost n .

In Phase 9, the first level costs n^2 , then the next levels become smaller and smaller.

13. Why There Are $\log n$ Levels

The input keeps being divided by 2:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

After k levels, the input size is:

$n / 2^k$

The recursion stops when the input reaches 1 :

$n / 2^k = 1$

Solve this equation:

$n / 2^k = 1$
 $n = 2^k$
 $k = \log_2(n)$

So the number of levels is:

$\log_2(n)$

In Big O notation, we usually write:

$\log n$

14. Why the Final Answer Is $O(n^2)$

Now combine the level costs:

$$n^2 + n^2/2 + n^2/4 + n^2/8 + \dots$$

Factor out n^2 :

$$n^2(1 + 1/2 + 1/4 + 1/8 + \dots)$$

The inside part is a geometric series:

$$1 + 1/2 + 1/4 + 1/8 + \dots$$

This sum approaches 2.

So:

$$n^2(1 + 1/2 + 1/4 + 1/8 + \dots) \approx n^2 \times 2$$

That gives:

$$2n^2$$

Big O ignores constant numbers.

So:

$$O(2n^2) = O(n^2)$$

Final answer:

$$T(n) = O(n^2)$$

15. Why the Answer Is Not $O(n \log n)$

A beginner may think:

There are two half-size recursive calls, so the answer must be $O(n \log n)$.

That was true in Phase 8:

$$T(n) = 2T(n/2) + n$$

But Phase 9 has:

$$T(n) = 2T(n/2) + n^2$$

The extra work is larger.

In Phase 8, each level cost:

$$n$$

and there were $\log n$ levels:

$$n + n + n + \dots = n \log n$$

In Phase 9, the levels cost:

$$n^2 + n^2/2 + n^2/4 + \dots$$

This sum is about:

$$2n^2$$

So the answer is:

$$O(n^2)$$

not:

$$O(n \log n)$$

16. Why the Answer Is Not $O(2^n)$

A beginner may also think:

There are two recursive calls, so the answer must be exponential.

That is not always true.

Two recursive calls become exponential when the input decreases slowly, like this:

$$T(n) = 2T(n - 1) + 1$$

That recurrence has many levels:

$$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow n - 3 \rightarrow \dots \rightarrow 0$$

So the tree becomes very deep.

But in Phase 9:

$$T(n) = 2T(n / 2) + n^2$$

The input shrinks quickly:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

So there are only $\log n$ levels.

That is why the answer is not exponential.

17. Successive Substitution Method

Now we solve the recurrence formally.

We solve:

$$T(n) = 2T(n / 2) + n^2$$

Step 1: Start with the recurrence

$$T(n) = 2T(n / 2) + n^2$$

Step 2: Substitute $T(n / 2)$

Using the same recurrence rule:

$$T(n / 2) = 2T(n / 4) + (n / 2)^2$$

Substitute this into the original recurrence:

$$T(n) = 2[2T(n / 4) + (n / 2)^2] + n^2$$

Simplify:

$$T(n) = 4T(n / 4) + 2(n^2 / 4) + n^2$$

$$T(n) = 4T(n / 4) + n^2/2 + n^2$$

Write 4 as 2^2 :

$$T(n) = 2^2T(n / 4) + n^2 + n^2/2$$

Step 3: Substitute again

Using the same recurrence rule:

$$T(n / 4) = 2T(n / 8) + (n / 4)^2$$

Substitute:

$$T(n) = 2^2[2T(n / 8) + (n / 4)^2] + n^2/2 + n^2$$

Simplify:

$$T(n) = 2^3T(n / 8) + n^2/4 + n^2/2 + n^2$$

Step 4: See the pattern

After 1 step:

$$T(n) = 2T(n / 2) + n^2$$

After 2 steps:

$$T(n) = 2^2T(n / 4) + n^2 + n^2/2$$

After 3 steps:

$$T(n) = 2^3T(n / 8) + n^2 + n^2/2 + n^2/4$$

After k steps:

$$T(n) = 2^kT(n / 2^k) + n^2(1 + 1/2 + 1/4 + \dots + 1/2^{(k-1)})$$

This is the general pattern.

18. Reaching the Base Case

The base case is:

$$T(1) = 1$$

So we want the input inside $T(\dots)$ to become 1 .

The recursive part is:

$$T(n / 2^k)$$

We want:

$$n / 2^k = 1$$

Now solve this equation:

$$\begin{aligned} n / 2^k &= 1 \\ n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

So the recursion reaches the base case after:

$$\log_2(n)$$

levels.

19. Substitute Back

The pattern is:

$$T(n) = 2^k T(n / 2^k) + n^2(1 + 1/2 + 1/4 + \dots + 1/2^{(k-1)})$$

At the base case:

$$n / 2^k = 1$$

So:

$$T(n / 2^k) = T(1)$$

Then:

$$T(n) = 2^k T(1) + n^2(1 + 1/2 + 1/4 + \dots)$$

Since:

$$T(1) = 1$$

We get:

$$T(n) = 2^k + n^2(1 + 1/2 + 1/4 + \dots)$$

But we know:

$$2^k = n$$

And:

$$1 + 1/2 + 1/4 + \dots \approx 2$$

So:

$$T(n) = n + 2n^2$$

The biggest term is:

$$n^2$$

Therefore:

$$T(n) = O(n^2)$$

20. Full Successive Substitution Solution in One Place

Start:

$$T(n) = 2T(n / 2) + n^2$$

Substitute once:

$$T(n) = 2[2T(n / 4) + (n / 2)^2] + n^2$$

$$T(n) = 2^2T(n / 4) + n^2/2 + n^2$$

Substitute again:

$$T(n) = 2^3T(n / 8) + n^2/4 + n^2/2 + n^2$$

Pattern:

$$T(n) = 2^kT(n / 2^k) + n^2(1 + 1/2 + 1/4 + \dots + 1/2^{(k-1)})$$

Use the base case:

$$n / 2^k = 1$$

Solve:

$$\begin{aligned} n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

Substitute into the pattern:

$$T(n) = 2^kT(1) + n^2(1 + 1/2 + 1/4 + \dots)$$

Since $T(1) = 1$:

$$T(n) = 2^k + n^2(1 + 1/2 + 1/4 + \dots)$$

Since $2^k = n$:

$$T(n) = n + n^2(1 + 1/2 + 1/4 + \dots)$$

The geometric series approaches 2 :

$$T(n) = n + 2n^2$$

Final Big O:

$$T(n) = O(n^2)$$

21. Beginner-Friendly Meaning of the Solution

The recurrence:

$$T(n) = 2T(n / 2) + n^2$$

means:

Do n^2 work now.
Then solve two smaller problems.
Each smaller problem has size $n / 2$.

The recursion tree gets wider:

```
1 call
2 calls
4 calls
8 calls
...
```

But the input size gets smaller:

n
n/2
n/4
n/8
...
1

The total work by level gets smaller:

n^2
 $n^2/2$
 $n^2/4$
 $n^2/8$
...

So most of the work is near the top of the tree.

The total is about:

$2n^2$

Final result:

$O(n^2)$

22. Division Master Theorem

Phase 9 also introduces the **division master theorem**.

The master theorem is a shortcut for recurrences of this form:

$$T(n) = aT(n/b) + f(n)$$

It helps us solve dividing recurrences faster without doing full substitution every time.

23. What a , b , and $f(n)$ Mean

In the recurrence:

$$T(n) = aT(n / b) + f(n)$$

Symbol	Meaning	Beginner meaning
a	number of recursive calls	How many smaller problems are created?
b	division factor	How much smaller is each problem?
f(n)	extra work outside recursion	How much work is done before/after recursion?

Example:

$$T(n) = 2T(n / 2) + n^2$$

Here:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n^2 \end{aligned}$$

24. Main Idea of the Division Master Theorem

The master theorem compares two things:

1. The growth caused by the recursive calls.
2. The growth caused by the extra work **f(n)**.

The recursive-call growth is measured by:

$$n^{(\log_b a)}$$

The extra work is usually written as:

$$f(n) = n^k$$

Then we compare:

$$\log_b(a)$$

with:

k

25. How to Find k

If:

$$f(n) = n^k$$

then k is the power of n.

Examples:

f(n)	Written as	k
1	n^0	0
n	n^1	1
n^2	n^2	2
n^3	n^3	3

For Phase 9:

$$f(n) = n^2$$

So:

$$k = 2$$

26. The Three Master Theorem Cases

For:

$$T(n) = aT(n/b) + f(n)$$

and:

$$f(n) = n^k$$

compare:

$$\log_b(a)$$

with:

$$k$$

Case 1: Recursive Calls Dominate

Case 1 happens when:

$$\log_b(a) > k$$

This means the recursive-call part grows faster than the extra work.

The answer is:

$$T(n) = O(n^{\log_b a})$$

Example

$$T(n) = 2T(n/2) + 1$$

Find the parts:

$$\begin{aligned}a &= 2 \\b &= 2 \\f(n) &= 1\end{aligned}$$

Since:

$$1 = n^0$$

we have:

$$k = 0$$

Now compute:

$$\log_b(a) = \log_2(2) = 1$$

Compare:

$$1 > 0$$

So Case 1 applies.

Final answer:

$$T(n) = O(n)$$

This matches Phase 7:

$$T(n) = 2T(n/2) + 1 = O(n)$$

Case 2: Balanced Growth

Case 2 happens when:

$$\log_b(a) = k$$

This means the recursive-call growth and the extra work are balanced.

The answer is:

$$T(n) = O(f(n) \log n)$$

If $f(n) = n^k$, this can also be written as:

$$T(n) = O(n^k \log n)$$

Example

$$T(n) = 2T(n/2) + n$$

Find the parts:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n \end{aligned}$$

Since:

$$n = n^1$$

we have:

$$k = 1$$

Now compute:

$$\log_b(a) = \log_2(2) = 1$$

Compare:

$$1 = 1$$

So Case 2 applies.

Final answer:

$$T(n) = O(n \log n)$$

This matches Phase 8 and merge sort.

Case 3: Extra Work Dominates

Case 3 happens when:

$$\log_b(a) < k$$

This means the extra work grows faster than the recursive-call part.

The answer is:

$$T(n) = O(f(n))$$

Example

$$T(n) = 2T(n / 2) + n^2$$

Find the parts:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n^2 \end{aligned}$$

Since:

$$n^2 = n^2$$

we have:

$$k = 2$$

Now compute:

$$\log_b(a) = \log_2(2) = 1$$

Compare:

$$1 < 2$$

So Case 3 applies.

Final answer:

$$T(n) = O(n^2)$$

This is the main Phase 9 result.

27. Master Theorem Summary Table

Case	Condition	Meaning	Final answer
Case 1	$\log_b(a) > k$	Recursive calls dominate	$O(n^{\log_b a})$
Case 2	$\log_b(a) = k$	Recursive calls and extra work balance	$O(f(n) \log n)$
Case 3	$\log_b(a) < k$	Extra work dominates	$O(f(n))$

28. Applying the Master Theorem to Phase 9

Solve:

$$T(n) = 2T(n/2) + n^2$$

Step 1: Match the form

General form:

$$T(n) = aT(n/b) + f(n)$$

Given recurrence:

$$T(n) = 2T(n/2) + n^2$$

So:

$$\begin{aligned}a &= 2 \\b &= 2 \\f(n) &= n^2\end{aligned}$$

Step 2: Find k

Since:

$$f(n) = n^2$$

we get:

$$k = 2$$

Step 3: Compute $\log_b(a)$

$$\log_b(a) = \log_2(2) = 1$$

Step 4: Compare

$$\begin{aligned}\log_b(a) &= 1 \\k &= 2\end{aligned}$$

Since:

$$1 < 2$$

this is Case 3.

Step 5: Apply Case 3

Case 3 says:

$$T(n) = O(f(n))$$

Since:

$$f(n) = n^2$$

Final answer:

$$T(n) = O(n^2)$$

29. Simple Memory Trick

For:

$$T(n) = 2T(n / 2) + n^2$$

remember this sentence:

Two half-size recursive calls try to grow the tree,
but the n^2 work at the top is so large that it dominates everything.

The work by level is:

n^2
 $n^2/2$
 $n^2/4$
 $n^2/8$
...

The first level is the biggest.

So the answer stays:

$$O(n^2)$$

30. Time Complexity

The time complexity is:

$$O(n^2)$$

Why?

Because the total work is:

$$n^2 + n^2/2 + n^2/4 + n^2/8 + \dots$$

This series approaches:

$$2n^2$$

And Big O ignores constants:

$$O(2n^2) = O(n^2)$$

So:

$$\text{Time complexity} = O(n^2)$$

31. Stack Space Complexity

The stack space complexity is:

$$O(\log n)$$

Why?

Even though the function creates many total calls, the computer does not keep all calls active at the same time.

The deepest active path looks like this:

```
algorithm(n)
algorithm(n / 2)
algorithm(n / 4)
algorithm(n / 8)
```

```
...  
algorithm(1)
```

The length of this path is:

$\log_2(n)$

So the stack space is:

$O(\log n)$

Important difference

Total work = $O(n^2)$
Stack depth = $O(\log n)$

The total work counts all calls and all loop work.

The stack space counts only the deepest active path at one time.

32. Time Complexity vs Stack Space

For Phase 9:

Recurrence	Time complexity	Stack space
$T(n) = 2T(n/2) + n^2$	$O(n^2)$	$O(\log n)$

Why time is $O(n^2)$:

$$n^2 + n^2/2 + n^2/4 + \dots = O(n^2)$$

Why stack space is $O(\log n)$:

The deepest path keeps dividing n by 2 until reaching 1.

33. Comparison with Earlier Dividing Phases

Feature	Phase 5	Phase 6	Phase 7	Phase 8	Phase 9
Recurrence	$T(n) = T(n/2) + 1$	$T(n) = T(n/2) + n$	$T(n) = 2T(n/2) + 1$	$T(n) = 2T(n/2) + n$	$T(n) = 2T(n/2) + n^2$
Recursive calls	1	1	2	2	2
Input change	$n/2$	$n/2$	$n/2$	$n/2$	$n/2$
Extra work	constant	linear	constant	linear	quadratic
Main work idea	$1 + 1 + \dots$	$n + n/2 + \dots$	$1 + 2 + 4 + \dots$	$n + n + \dots$	$n^2 + n^2/2 + \dots$
Number of levels	$\log n$	$\log n$	$\log n$	$\log n$	$\log n$
Time complexity	$O(\log n)$	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
Stack space	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Key lesson

All these phases divide the input by 2, so all have $\log n$ stack depth.

But their time complexities are different because:

1. The number of recursive calls may differ.
2. The work done at each call may differ.

34. Important Terms in Phase 9

Dividing Function

A recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

Division Recurrence

A recurrence where the recursive part contains division.

Example:

$$T(n) = 2T(n / 2) + n^2$$

Quadratic Work

Work that grows like `n × n`.

Example:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << i;  
    }  
}
```

This is:

$$O(n^2)$$

Two Recursive Calls

The function calls itself twice.

Example:

```
algorithm(n / 2);  
algorithm(n / 2);
```

This gives:

$$2T(n / 2)$$

Base Case

The condition where recursion stops.

In Phase 9:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Recursion Tree

A tree diagram that shows how recursive calls split.

For Phase 9:

```
      n
     /
    n/2  n/2
   / \  /
  n/4 n/4 n/4 n/4
```

Each node represents one function call.

Geometric Series

A geometric series is a sequence where each term is multiplied by the same ratio.

In Phase 9, the work series is:

```
n2 + n2/2 + n2/4 + n2/8 + ...
```

The ratio is:

$$1/2$$

The sum is about:

$$2n^2$$

So the final time is:

$$O(n^2)$$

Division Master Theorem

A shortcut for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

It compares:

$$\log_b(a)$$

with the power of n in $f(n)$.

35. Common Beginner Mistakes

Mistake 1: Saying the answer is $O(n \log n)$

Wrong idea:

$$T(n) = 2T(n/2) + n^2 \text{ is } O(n \log n)$$

Correct idea:

$$T(n) = 2T(n/2) + n^2 \text{ is } O(n^2)$$

Why?

Because the extra work is n^2 , not n .

Mistake 2: Forgetting that nested loops multiply

Wrong idea:

Two loops means $O(n + n)$

Correct idea:

Nested loops mean $O(n \times n) = O(n^2)$

This code:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << i;  
    }  
}
```

is:

$O(n^2)$

not:

$O(n)$

Mistake 3: Thinking two recursive calls always means exponential time

Wrong idea:

Two recursive calls always give $O(2^n)$

Correct idea:

Two recursive calls can be exponential, linear, $n \log n$, or n^2 depending on how the input changes and how much extra work happens.

Examples:

Recurrence	Result
$T(n) = 2T(n - 1) + 1$	$O(2^n)$
$T(n) = 2T(n / 2) + 1$	$O(n)$
$T(n) = 2T(n / 2) + n$	$O(n \log n)$
$T(n) = 2T(n / 2) + n^2$	$O(n^2)$

Mistake 4: Applying the master theorem without comparing

Do not just memorize the number of recursive calls.

Always find:

a , b , $f(n)$, $\log_b(a)$, k

Then compare:

$\log_b(a)$ with k

Mistake 5: Forgetting stack space

The time complexity is:

$O(n^2)$

But the stack space is:

$O(\log n)$

Why?

Because the deepest active path divides by 2 each time.

36. Step-by-Step Checklist for Phase 9 Problems

When you see a recurrence or recursive code, follow these steps.

Step 1: Check if it is a dividing recurrence

Look for:

```
T(n / 2), T(n / 3), T(n / b)
```

If the input is divided, use division-recurrence thinking.

Step 2: Count recursive calls

If the code has:

```
algorithm(n / 2);  
algorithm(n / 2);
```

then:

```
a = 2
```

Step 3: Find the division factor

If the recursive call is:

```
algorithm(n / 2);
```

then:

```
b = 2
```

Step 4: Find the extra work $f(n)$

Look outside the recursive calls.

Examples:

Extra work	$f(n)$
print once	1
one loop from 1 to n	n
nested loops from 1 to n	n^2

For Phase 9, nested loops give:

$$f(n) = n^2$$

Step 5: Write the recurrence

For Phase 9 code:

$$T(n) = 2T(n/2) + n^2$$

Step 6: Use level-by-level thinking or master theorem

Level-by-level work:

$$n^2 + n^2/2 + n^2/4 + \dots = O(n^2)$$

Master theorem:

$$\begin{aligned}a &= 2 \\b &= 2 \\f(n) &= n^2 \\k &= 2 \\\log_2(2) &= 1 \\1 &< 2\end{aligned}$$

Case 3
 $O(n^2)$

37. Practice Examples

Example 1

Solve:

$$T(n) = 2T(n / 2) + n^2$$

Solution:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n^2 \\ k &= 2 \\ \log_2(2) &= 1 \end{aligned}$$

Compare:

$$1 < 2$$

So Case 3 applies.

Answer:

$$O(n^2)$$

Example 2

Solve:

$$T(n) = 2T(n / 2) + n$$

Solution:

$a = 2$
 $b = 2$
 $f(n) = n$
 $k = 1$
 $\log_2(2) = 1$

Compare:

$1 = 1$

So Case 2 applies.

Answer:

$O(n \log n)$

Example 3

Solve:

$T(n) = 2T(n / 2) + 1$

Solution:

$a = 2$
 $b = 2$
 $f(n) = 1 = n^0$
 $k = 0$
 $\log_2(2) = 1$

Compare:

$1 > 0$

So Case 1 applies.

Answer:

$O(n)$

38. Complete Coding Program for Phase 9

```
#include <iostream>
using namespace std;

// This function does quadratic work,
// then calls itself twice with n / 2.
void algorithm(int n) {
    if (n > 1) {
        // Quadratic work:  $O(n^2)$ 
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                cout << i << " ";
            }
        }

        cout << "| ";

        // Two recursive calls on half the input
        algorithm(n / 2);
        algorithm(n / 2);
    }
}

int main() {
    int n = 4;

    cout << "Output: ";
    algorithm(n);
    cout << endl;

    return 0;
}
```

Recurrence

$$T(n) = 2T(n / 2) + n^2$$
$$T(1) = 1$$

Time complexity

$O(n^2)$

Stack space

$O(\log n)$

39. Phase 9 Final Summary

The main recurrence is:

$$T(n) = 2T(n / 2) + n^2$$

Meaning:

$2T(n / 2)$ = two recursive calls, each half size
 n^2 = nested-loop work
 $T(1) = 1$ = base case

The level work is:

$$n^2 + n^2/2 + n^2/4 + n^2/8 + \dots$$

This sum is about:

$$2n^2$$

So the final time is:

$$O(n^2)$$

The stack space is:

$$O(\log n)$$

because the deepest recursive path is:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

The division master theorem shortcut uses:

$$T(n) = aT(n / b) + f(n)$$

For Phase 9:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n^2 \\ k &= 2 \\ \log_2(2) &= 1 \end{aligned}$$

Since:

$$1 < 2$$

Case 3 applies:

$$T(n) = O(f(n)) = O(n^2)$$

Final memory sentence

Two recursive calls on half the input create a tree,
but the n^2 work is so large that it dominates the total.
Therefore, $T(n) = 2T(n / 2) + n^2$ gives $O(n^2)$.